LeetCode 152 — Maximum Product Subarray

1. Problem Title & Link

Title: LeetCode 152: Maximum Product Subarray

Link: https://leetcode.com/problems/maximum-product-subarray/

2. Problem Statement (Short Summary)

Given an integer array nums, find the contiguous subarray that has the largest product and return that product.

Unlike Maximum Sum Subarray, multiplication behaves differently because:

Negative × Negative = Positive

This makes the problem more interesting.

3. Examples (Input → Output)

Example 1

Input:

nums = [2,3,-2,4]

Output:

6

Explanation:

Subarray:

[2,3]

Product:

2 × 3 = 6

Example 2

Input:

nums = [-2,0,-1]

Output:

0

Explanation:

[-2,-1]

is not contiguous because of zero.

Maximum product is:

0

Example 3

Input:

nums = [-2,3,-4]

Output:

24

Explanation:

(-2) × 3 × (-4)

= 24

4. Constraints

1 <= nums.length <= 20000

-10 <= nums[i] <= 10

Important:

Subarray must be contiguous.

5. Core Concept (Pattern / Topic)

Kadane's Algorithm Variant

Dynamic Programming

This is an advanced extension of LC 53 Maximum Subarray.

6. Thought Process (Step-by-Step Explanation)

Brute Force Approach

Generate every subarray.

Compute product.

Track maximum.

Example:

[2,3,-2,4]

[2]

[2,3]

[2,3,-2]

[2,3,-2,4]

[3]

[3,-2]

...

Time Complexity:

$O(n^2)$

Too slow.

Why Normal Kadane Fails

For sums:

Negative values are usually bad.

For products:

Negative values can become very good.

Example:

[-2,3,-4]

At index 0:

max = -2

At index 1:

max = 3

At index 2:

-2 × 3 × -4 = 24

Suddenly a huge positive value appears.

Key Insight

At every position we must maintain:

Maximum Product Ending Here

currentMax

Minimum Product Ending Here

currentMin

Why minimum?

Because:

A negative minimum

×

negative number

=

huge positive maximum

7. Visual / Intuition Diagram (ASCII)

```
Example:
[-2, 3, -4]
After processing:
currentMax = 3
```

```
currentMin = -6
See:
-4
Multiply:
currentMax × -4 = -12

currentMin × -4 = 24
Suddenly:
24
becomes the new maximum.
That's why we track both values.
```

8. Pseudocode

```
currentMax = nums[0]

currentMin = nums[0]

answer = nums[0]

for each remaining number

    if number < 0

        swap currentMax and currentMin

    currentMax =
        max(number,
            currentMax * number)

    currentMin =
        min(number,
            currentMin * number)

    answer =
        max(answer, currentMax)

return answer
```

9. Code Implementation

Python

```
class Solution:
    def maxProduct(self, nums):

        currentMax = nums[0]
        currentMin = nums[0]

        answer = nums[0]
```

```
        for num in nums[1:]:

            if num < 0:
                currentMax, currentMin = currentMin, currentMax

            currentMax = max(num, currentMax * num)

            currentMin = min(num, currentMin * num)

            answer = max(answer, currentMax)

        return answer
```

Java

```
class Solution {

    public int maxProduct(int[] nums) {

        int currentMax = nums[0];
        int currentMin = nums[0];

        int answer = nums[0];

        for(int i = 1; i < nums.length; i++) {

            int num = nums[i];

            if(num < 0) {

                int temp = currentMax;
                currentMax = currentMin;
                currentMin = temp;
            }

            currentMax =
                Math.max(num,
                         currentMax * num);

            currentMin =
                Math.min(num,
                         currentMin * num);

            answer =
                Math.max(answer,
                         currentMax);
        }

        return answer;
```

```
        }
    }
```

10. Time & Space Complexity

Time Complexity

O(n)

Single traversal.

Space Complexity

O(1)

Only variables used.

11. Common Mistakes / Edge Cases

Mistake 1

Tracking only maximum.

Need:

maximum

and

minimum

Mistake 2

Forgetting swap when number is negative.

Mistake 3

Ignoring zero.

Example:

[-2,0,-1]

Zero resets product chain.

12. Detailed Dry Run

Input:

nums = [-2,3,-4]

Initial:

currentMax	currentMin	answer
-2	-2	-2

Process 3

Value	currentMax	currentMin	answer
3	3	-6	3

Process -4

Negative number.

Swap:

currentMax = -6

currentMin = 3

Update:

currentMax

= max(-4, -6×-4)

= 24

currentMin

= min(-4, 3×-4)

= -12

Answer:

24

13. Common Use Cases (Real-Life / Interview)

Growth Multipliers

Examples:

Investment growth

Population growth

Compound multipliers

Signal amplification

Products matter more than sums.

14. Common Traps (Important!)

Trap 1

Treating it like LC 53.

This is not a sum problem.

Trap 2

Ignoring negative numbers.

Trap 3

Not swapping max and min.

This is the most common interview mistake.

15. Builds To (Related LeetCode Problems)

LC 53 Maximum Subarray

↓

LC 152 Maximum Product Subarray

↓

LC 918 Maximum Circular Subarray

↓

LC 1749 Maximum Absolute Sum

Kadane progression.

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	O(1)	Too slow
DP Arrays	O(n)	O(n)	Easy to understand
Kadane Variant	O(n)	O(1)	Optimal

Approach 1: DP Arrays

Store:

maxProduct[i]

minProduct[i]

for every index.

Works well but uses extra space.

Approach 2: Kadane Variant

Keep only current values.

Most efficient.

17. Why This Solution Works (Short Intuition)

A negative number can turn:

smallest negative

into

largest positive

Therefore we maintain both:

currentMax

currentMin

At each step we update both possibilities and keep track of the best product seen so far.

18. Variations / Follow-Up Questions

Follow-Up 1

Return the actual subarray.

Follow-Up 2

Find minimum product subarray.

Follow-Up 3

Maximum product circular subarray.

Much harder.

Follow-Up 4

What if multiplication overflows?

Use larger numeric types.

Pattern Learned

Kadane Variant

Track Maximum Product

+

Track Minimum Product

↓

Negative Numbers Can Flip Signs

↓

Maximum Product Subarray

This is one of the most important medium-level Dynamic Programming / Kadane interview problems because it teaches why maintaining both extremes (max and min) is necessary.